

打造 Claude 技能完 全指南



Claude

目录

引言	3
基础要点	4
规划与设计	7
测试与迭代	14
发布与分享	18
模式与故障排除	21
资源与参考资料	28

引言

技能是一组指令 —— 封装为一个简单的文件夹 —— 用于教 Claude 如何处理特定任务或 workflows。技能是针对你的特定需求定制 Claude 的最强大方式之一。无需在每次对话中重新说明你的偏好、流程和领域专业知识，技能能让你只需教会 Claude 一次，就能在每次使用中获益。

当你拥有可重复的工作流程时，技能会发挥强大作用：根据规范生成前端设计、采用一致的方法论开展研究、创建符合团队风格指南的文档，或是统筹多步骤流程。这些技能能与 Claude 内置的代码执行、文档创建等功能完美配合。对于那些构建 MCP 集成的人来说，技能又增添了一层强大的功能，助力将原始的工具访问转化为可靠、优化的工作流程。

本指南涵盖了打造实用技能所需的全部知识 —— 从规划、构建框架，到测试与发布。无论你是为自己、团队还是社区打造技能，都能在书中找到实用的模式与真实案例。

你将学到：

- 技能结构的技术要求和最佳实践
- 独立技能与 MCP 增强型 workflows 的模式
- 我们在不同使用场景验证过的有效模式
- 如何测试、迭代和发布你的技能

适用人群：

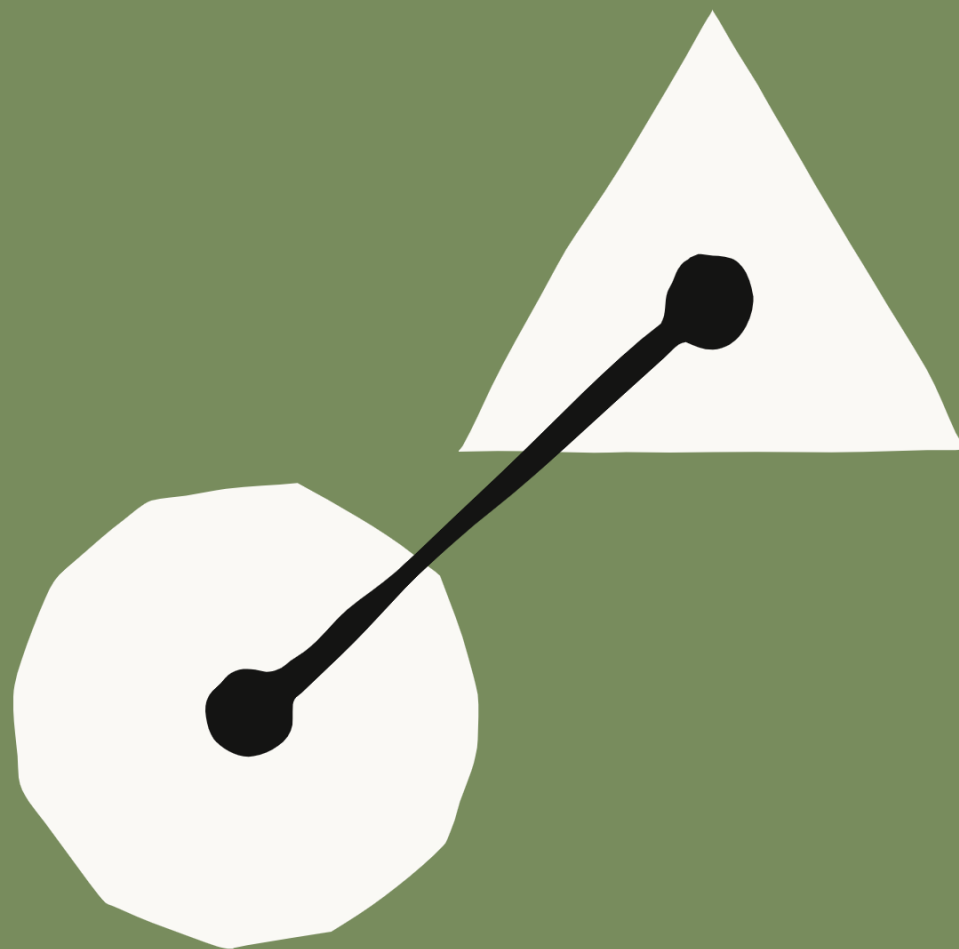
- 希望 Claude 始终遵循特定 workflows 的开发者
- 希望 Claude 遵循特定 workflow 的高级用户
- 希望在整个组织内统一 Claude 使用方式的团队

本指南的两条学习路径

想打造独立技能？请专注于基础、规划与设计以及 1-2 类技能。想强化 MCP 集成？“技能 + MCP” 部分和 3 类技能适合你。两种路径的技术要求相同，你可以选择与自身用例相关的内容。

你将从本指南中收获：学完本指南后，你就能一次性打造出一个可用的技能。使用技能创建工具构建并测试你的第一个可用技能，预计需要 15 至 30 分钟。

让我们开始吧。



第 1 章

基础

基础

什么是技能？

技能是一个包含以下内容的文件夹：

- SKILL.md（必需）：包含 YAML 前置内容的 Markdown 格式说明文档
- scripts/（可选）：可执行代码（Python、Bash 等）
- references/（可选）：按需加载的文档
- assets/（可选）：输出内容中使用的模板、字体、图标

核心设计原则

渐进式披露

技能采用二级系统。

- 第一层级（YAML 前置内容）：始终加载在 Claude 的系统提示中。只需提供足够信息，让 Claude 明确每种技能的使用时机，无需将所有信息加载到上下文。
- 第二层（SKILL.md 主体）：当 Claude 认为该技能与当前任务相关时加载。包含完整的指令和指导内容。
- 第三级（链接文件）：捆绑在技能目录内的额外文件，Claude 可选择在需要时导航和查找这些文件。

这种渐进式披露的方式能在保持专业能力的同时最大限度减少令牌消耗。

组合性

Claude 可以同时加载多项技能。你的技能应能与其他技能良好配合，而不应假定自身是唯一可用的功能。

便携性

Claude.ai、Claude Code 和 API 中的技能功能完全一致。只要环境支持技能所需的任何依赖项，创建一次技能即可在所有平台上无修改地运行。

面向 MCP 开发者：技能 + 连接器

💡 不使用 MCP 构建独立技能？跳转到规划与设计 —— 你之后随时可以回到这里。

如果你已经有了一个可用的 MCP 服务器，那么最难的部分你已经完成了。技能是在此之上的知识层 —— 它会捕获你已有的工作流程和最佳实践，让 Claude 能够始终如一地应用这些内容。

厨房类比

MCP 就像专业的厨房：能获取工具、食材和设备。

技能就好比是食谱：提供了创造有价值成果的分步指导。

它们共同帮助用户完成复杂任务，无需自己弄清楚每一个步骤。

它们如何协同工作。

MCP (Connectivity)	Skills (Knowledge)
Connects Claude to your service (Notion, Asana, Linear, etc.)	Teaches Claude how to use your service effectively
Provides real-time data access and tool invocation	Captures workflows and best practices
What Claude can do	How Claude should do it

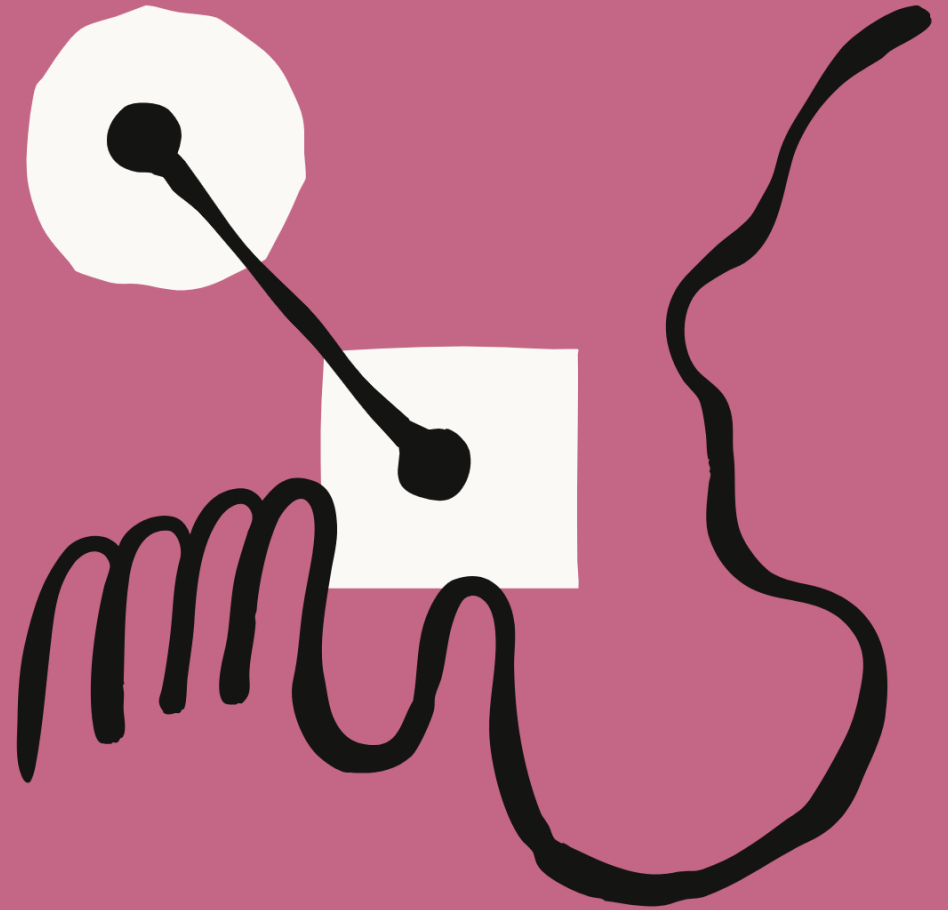
这对你的 MCP 用户有何重要意义

缺乏技能时。

- 用户连接了你的 MCP，但不知道下一步该做什么
- 支持简单询问 “如何通过你的集成完成 X 操作”
- 每次对话都从零开始
- 结果不一致。因为用户每次的提示词都不同
- 用户将问题归咎于你的连接器，但实际真正的问题是工作流指引缺失

且各技能时。

- 预构建的工作流会在需要时自动激活
- 一致且可靠的工具使用
- 每一次交互中都融入了最佳实践
- 降低你的集成的使用门槛



第 2 章

规划与设计

规划与设计

从用例入手

在编写任何代码之前，确定你的技能应支持的 2-3 个具体用例。

优质用例定义。

```
Use Case: Project Sprint Planning
Trigger: User says "help me plan this sprint" or "create sprint tasks"
Steps:
1. Fetch current project status from Linear (via MCP)
2. Analyze team velocity and capacity
3. Suggest task prioritization
4. Create tasks in Linear with proper labels and estimates
Result: Fully planned sprint with tasks created
```

问问自己。

- 用户想要完成什么？
- 这需要哪些多步骤工作流？
- 需要哪些工具（内置工具还是 MCP 工具？）
- 应融入哪些领域知识或最佳实践？

常见技能用例类别

在 Anthropic，我们观察到了三种常见的用例：

类别 1：文档与资产创建

用途：用于生成一致、高质量的输出内容，包括文档、演示文稿、应用程序、设计方案、代码等。

实际示例：前端设计技能（另请参阅适用于 docx、pptx、xlsx 和 ppt 的技能）

“打造具有卓越设计品质的、可投入生产的独特前端界面。适用于构建网页组件、页面、设计稿、海报或应用程序。”

关键技巧。

- 嵌入式风格指南与品牌标准
- 用于生成一致输出的模板结构
- 最终定稿前的质量检查清单
- 无需外部工具 —— 利用 Claude 的内置功能

类别 2：工作流自动化

用途：适用于可从统一方法中获益的多步骤流程，包括跨多个 MCP 服务器的协调工作。

实际示例，技能创建器技能

“创建新技能的交互式指南。引导用户完成用例定义、前置内容生成、指令编写和验证流程。”

核心技巧：

- 带验证关卡的分步工作流
- 通用结构模板
- 内置的审核与改进建议
- 迭代优化循环

第 3 类：MCP 增强

用于：工作流指南，以增强 MCP 服务器提供的工具访问。

实际示例，哨兵代码审查技能（来自哨兵）

“使用 Sentry 通过其 MCP 服务器的错误监控数据自动分析和修复 GitHub 拉取请求中检测到的错误。”

关键技术：

- 按顺序协调多个 MCP 调用

嵌入领域专业知识

- 提供用户原本需要自行明确的上下文信息
- 常见 MCP 问题的错误处理

定义成功标准

你如何判断你的技能是否在正常运行？

这些是具有前瞻性的目标 —— 只是大致基准，而非精确阈值。追求严谨的同时，也需接受评估过程中会带有一定的主观感受成分。我们正积极制定更完善的衡量指导标准与配套工具。

量化指标：

- 技能在 90% 的相关查询中触发
 - 测量方法：运行 10-20 个应触发你的技能的测试查询。跟踪它自动加载的次数与需要显式调用的次数。
- 在 X 次工具调用内完成工作流程
 - 衡量方法：对比启用该技能与未启用该技能时的同一任务。统计工具调用次数和消耗的总令牌数。
- 每个工作流无失败的 API 调用
 - 衡量方法：在测试运行期间监控 MCP 服务器日志，跟踪重试率和错误代码。

定性指标：

- 用户无需向 Claude 询问下一步操作
 - 评估方法：测试过程中，记录需要重定向或澄清的频率。向测试用户收集反馈。
- 无需用户修正即可完成工作流程
 - 评估方法：将同一请求运行 3-5 次。对比输出结果的结构一致性和质量。
- 跨会话结果一致
 - 如何评估：新用户能否在极少指导的情况下，首次尝试就完成该任务？

技术要求

文件结构

关键规则

SKILL.md 命名规范

- 必须为 SKILL.md (即八十小写)
- 不接受任何变体 (如 SKILL MD, skill.md 等)

技能文件命名规范

- 技能名称规范化: notion-xxx-xxx-xxx
- 空格: 必须用下划线替换 (如 notion-xxx-xxx-xxx)
- 不要使用下划线: notion-xxx-xxx-xxx
- 不要使用大写字母: Notion-Development

主创建

- 不要在技能文件中包含 README.md 文件
- 所有文件结构 (SKILL.md 或 notion-xxx-xxx-xxx / 目录)

• 注意: 通过 GitHub 分发时, 你仍需要一个仓库级别的 README 文件供人工用户
本文档 请参考 《八条规则》

YAML 前置内容是 Claude 判断是否加载你的技能的依据

YAML 前置内容是 Claude 判断是否加载你的技能的依据。务必正确设置。

最低要求

技能文件命名规范

字段要求

技能名称

- 技能名称规范化
- 空格: 必须用下划线替换
- 不要使用下划线

技能描述

- 技能描述规范化
- 技能描述用下划线替换
- 技能描述用下划线替换

技能文件结构

- 技能文件结构

技能文件命名规范

- 技能文件命名规范

许可证（可选）。

- 活用于将技能开源的情况
- 常见：麻省理工学院、Apache-2.0

兼容性（可选）。

- 1-500 个字符
- 指示环境要求：例如目标产品、所需系统软件包、网络访问需求等

元数据（可选）。

- 任何白空 \ 键值对
- 建议面，作者，版本，mcp-server
- 示例。

```
yaml
metadata:
  author: ProjectHub
  version: 1.0.0 mcp-server: projecthub
```

安全限制

禁止出现在前置内容中。

- XML 尖括号 (< >)
- 名称中包含 “claude” 或 “anthropic” 的技能（保留）

原因：前置内容会出现在 Claude 的系统提示中，恶意内容可能借此注入指令。

撰写高效的技能

描述字段

根据 Anthropic 的工程博客所述：“此元数据…… 为 Claude 提供了足够的信息，使其能够知晓每项技能的使用时机，而无需将所有信息加载到上下文之中。” 这是渐进式披露的第一层级。

结构。

[What it does] + [When to use it] + [Key capabilities]

优质描述示例：

```
# Good - specific and actionable
description: Analyzes Figma design files and generates
developer handoff documentation. Use when user uploads .fig
files, asks for "design specs", "component documentation", or
"design-to-code handoff".
```

```
# Good - includes trigger phrases
description: Manages Linear project workflows including sprint
planning, task creation, and status tracking. Use when user
mentions "sprint", "Linear tasks", "project planning", or asks
to "create tickets".
```

```
# Good - clear value proposition
description: End-to-end customer onboarding workflow for
PayFlow. Handles account creation, payment setup, and
subscription management. Use when user says "onboard new
customer", "set up subscription", or "create PayFlow account".
```

糟糕描述的示例。

```
# Too vague
description: Helps with projects.

# Missing triggers
description: Creates sophisticated multi-page documentation
systems.

# Too technical, no user triggers
description: Implements the Project entity model with
hierarchical relationships.
```

撰写核心指令

在 frontmatter 之后，用 Markdown 编写实际的指令。

堆叠结构。

根据您的技能调整此模板。用您的特定内容替换括号内的部分。

```
---
name: your-skill
description: [...]
---

# Your Skill Name

## Instructions

### Step 1: [First Major Step]
Clear explanation of what happens.
```

```
Example:
```bash
python scripts/fetch_data.py --project-id PROJECT_ID
Expected output: [describe what success looks like]
```

(根据需要添加更多步骤)

## 示例

### 示例 1. 「常见场景」

用户说：“发起一场新的营销活动”

行动。

1. 通过 MCP 获取现有营销活动
2. 根据提供的参数创建新的营销活动

结果。已创建营销活动并附带确认链接

(根据需要添加更多示例)

## 故障排除

### 错误：「常见错误信息」

原因。「发生原因」

解决方案。「解决方法」

(根据需要添加更多错误案例)

## 指令最佳实践

具体且可执行

✅ 好的.

```
Run `python scripts/validate.py --input {filename}` to check data format.
```

If validation fails, common issues include:

- Missing required fields (add them to the CSV)
- Invalid date formats (use YYYY-MM-DD)

❌ 错误示

```
Validate the data before proceeding.
```

包含错误处理

```
Common Issues
```

```
MCP Connection Failed
```

If you see "Connection refused":

1. Verify MCP server is running: Check Settings > Extensions
2. Confirm API key is valid
3. Try reconnecting: Settings > Extensions > [Your Service] > Reconnect

清晰引用捆绑的资源

```
Before writing queries, consult `references/api-patterns.md` for:
```

- Rate limiting guidance
- Pagination patterns
- Error codes and handling

采用渐进式披露

让 SKILL.md 聚焦核心说明。将详细文档移至 `references/` 目录并添加指向它的链接。（有关三级系统的工作原理，请参阅核心设计原则。）

### 第 3 章

# 测试与评估

# 测试与迭代

可根据你的需求，以不同严格程度对技能进行测试：

- 在 Claude.ai 中进行手动测试 - 直接运行查询并观察行为。快速迭代，无需设置。
- Claude Code 脚本化测试 - 自动化测试用例，以便在各项变更中进行可重复的验证
- 通过技能 API 进行程序化测试 - 构建评估套件，针对定义的测试集进行系统性运行。

选择符合你的质量要求和技能可见性的方法。一个供小团队内部使用的技能，其测试需求与部署给数千名企业用户的技能截然不同。

专业技巧：先聚焦单个任务进行迭代，再扩展范围

我们发现，最高效的技能创建者会针对单个具有挑战性的任务进行迭代，直到 Claude 成功完成，然后将这种成功的方法提炼为一项技能。这充分利用了 Claude 的上下文学习能力，且相比广泛测试能更快获得反馈。一旦你有了可用的基础框架，就可以扩展到多个测试用例以实现全面覆盖。

## 推荐的测试方法

根据早期经验，有效的技能测试通常涵盖三个方面：

### 1. 触发测试

目标：确保你的技能在正确的时间加载。

测试用例：

-  触发明显任务
-  对改写后的请求触发
-  不触发与无关主题相关的调用

示例测试套件：

Should trigger:

- "Help me set up a new ProjectHub workspace"
- "I need to create a project in ProjectHub"
- "Initialize a ProjectHub project for Q4 planning"

Should NOT trigger:

- "What's the weather in San Francisco?"
- "Help me write Python code"
- "Create a spreadsheet" (unless ProjectHub skill handles sheets)

## 2. 功能测试

目标，验证技能能否生成正确的输出。

测试用例。

- 生成有效输出
- API 调用成功
- 错误处理正常生效
- 覆盖边缘情况

示例。

```
Test: Create project with 5 tasks
Given: Project name "Q4 Planning", 5 task descriptions
When: Skill executes workflow
Then:
 - Project created in ProjectHub
 - 5 tasks created with correct properties
 - All tasks linked to project
 - No API errors
```

## 3. 性能对比

目标，证明该技能相比基准能提升效果。

使用定义成功标准中确定的指标。以下是一次对比可能呈现的样子。

基准对比。

```
Without skill:
- User provides instructions each time
- 15 back-and-forth messages
- 3 failed API calls requiring retry
- 12,000 tokens consumed
```

```
With skill:
- Automatic workflow execution
- 2 clarifying questions only
- 0 failed API calls
- 6,000 tokens consumed
```

## 创建技能创建技能

技能创建技能 —— 可通过 Claude.ai 的插件目录获取，或下载用于 Claude Code—— 能帮助你创建并迭代优化技能。如果你拥有一个 MCP 服务器，且清楚自己最核心的 2 到 3 个工作流程，那么你可以一次性完成一个实用技能的创建与测试 —— 通常只需 15 到 30 分钟。

创建技能。

- 根据自然语言描述生成技能
- 生成格式正确的带前置内容的 SKILL.md
- 建议触发短语和结构

审核技能。

- 标记常见问题（描述模糊、缺少触发词、结构问题）
- 识别潜在的过度触发或触发不足风险
- 根据技能的既定用途建议测试用例

迭代优化。

- 在使用你的技能并遇到边缘情况或失败情况后，将这些示例反馈给技能创建者
- 示例：“使用本次对话中确定的问题和解决方案，改进该技能处理 [特定边缘情况] 的方式”



使用它。

注：技能创建工具可帮助你设计和优化技能，但不会执行自动化测试套件，  
由于人工检查员仍须使用它。

## 基于反馈的迭代优化

技能目录会更新的内容，让技能工具下内容进行迭代。

触发不足的信号，

- 技能在应该加盐时未加盐
- 用户手动启用它
- 关于何时使用该技能的去盐咨询

解决方案：为描述添加更多细节与细微差别 —— 这可包含专门针对专业术语  
的词汇。

过度触发信号

- 技能针对不相干案例触发

固定该技能

- 对技能目的左右困惑

解决方案：添加负面触发词，提高表述精准度

其他问题

技能工具 2.0

使用技能

- 重新启用修正

解决方案：完善说明，添加错误处理机制

lun

m

第 4 章

# 分发与共享

# 分发与共享

技能让你的 MCP 集成更完善。当用户对比各类连接器时，具备技能的连接器能更快实现价值落地，让你相比仅支持 MCP 的同类产品更具优势。

## 当前分发模式（2026 年 1 月）

### 公共 MCP 市场中的技能发布

- 1. 技能所有者在 MCP 市场发布技能
- 2. 技能所有者在 MCP 市场发布技能
- 3. 技能所有者在 MCP 市场发布技能
- 4. 技能所有者在 MCP 市场发布技能

### 技能分发模式

- 管理员可在整个工作区部署技能（已于 2025 年 12 月 18 日推出）
- 自动更新
- 集中化管理

## 开放标准

我们已将智能体技能作为开放标准发布。与模型上下文协议（MCP）一样，我们认为技能应具备跨工具和跨平台的可移植性——同一技能在你使用 Claude 或其他人工智能平台时均可生效。不过，部分技能是为充分利用特定平台的功能而设计的；开发者可在技能的兼容性字段中对此进行说明。我们一直在与生态系统中的相关方合作推进该标准的制定，早期的采用情况也让我们倍感振奋。

## 通过 API 使用技能

对于程序化使用场景（例如构建利用技能的应用程序、智能体或自动化工作流），该 API 提供了对技能管理和执行的直接控制。

### 技能 API

- `/v1/skills` 端点用于列出和管理技能
  - 通过 `add_skill` 端点将技能添加到 MCP 服务器
  - 通过 Claude 控制台进行基本控制与管理
- 可与 Claude Agent SDK 配合用于构建自定义智能体

### 技能分发与更新策略

注意：API 中的技能需要 Code Execution Tool 测试版，该工具可提供技能运行所需的所有依赖。

有关部署技能，请参阅。

- 技能 API 快速入门
- 创建自定义技能
- Agent SDK 中的技能

### 部署技能安全

先将你的技能托管到 GitHub，创建一个公开代码仓库，编写清晰的 README（面向人类访问者 —— 这与你的技能文件夹相互独立，技能文件夹中不应包含 README.md 文件），并附上带截图的使用示例。随后在你的 MCP 文档中新增一个板块，链接到该技能，说明将两者结合使用的价值，并提供快速入门指南。

- 1. 开源技能的公共代码仓库
- 清晰的自建示例 和 今天状态说明
- 二. 项目主页截图

- 2. 在 MCP 文档中提供公共代码仓库
- 在 MCP 文档中添加技能链接
- 说明同时使用两者的价值
- 提供快速入门指南

3.

### 宣传你的技能

你如何描述你的技能，决定了用户能否理解其价值并真正尝试它。在介绍你的技能时 —— 无论是在自述文件、文档还是营销材料中 —— 请牢记以下原则。

#### 避免使用“开箱即用”

错误

“ProjectHub 技能让团队能在几秒钟内搭建起完整的项目工作空间 —— 包括页面、数据库和模板 —— 无需花 30 分钟进行手动设置。”

#### ✖ 错误

“ProjectHub 技能是一个包含 YAML 前端内容和 Markdown 说明的文件夹，用于调用我们的 SaaS 服务器工具。”

#### 避免 MCP 与技能的结合使用

“我们的 MCP 服务器让 Claude 能够访问你的 Linear 项目。我们的技能为 Claude 传授了你团队的冲刺规划工作流程。二者结合，可实现人工智能驱动的项目管理。”

!

第 5 章

# 模式与故障排除

# 模式与故障排除

这些模式源自早期采用者和内部团队所打造的技能。它们代表了我们观察到的行之有效的常见方法，而非规定性模板。

## 选择你的方法：先问题后工具 vs. 先工具后问题

可以把它想象成家得宝建材超市。你可能带着一个问题走进来——“我需要修理厨房橱柜”，然后店员会给你指明合适的工具。又或者你会挑选一把新电钻，然后询问如何用它来完成自己的具体工作。

技能的应用方式也是如此：

- 以问题为导向：“我需要搭建一个项目工作区” → 你的技能按正确顺序编排合适的 MCP 调用。用户只需描述目标结果，由技能负责处理工具操作。
- 工具优先：“我已连接 Notion MCP” → 你的技能向 Claude 传授最优工作流程和最佳实践。用户可访问；技能提供专业知识。

大多数技能都偏向某一个方向。了解哪种框架适合你的用例，有助于你在下方选择正确的模式。

## 模式 1：顺序 workflow 编排

适用场景：你的用户需要按特定顺序执行的多步骤流程。

示例结构：

```
Workflow: Onboard New Customer

Step 1: Create Account
Call MCP tool: `create_customer`
Parameters: name, email, company

Step 2: Setup Payment
Call MCP tool: `setup_payment_method`
Wait for: payment method verification

Step 3: Create Subscription
Call MCP tool: `create_subscription`
Parameters: plan_id, customer_id (from Step 1)

Step 4: Send Welcome Email
Call MCP tool: `send_email`
Template: welcome_email_template
```

关键技巧：

- 明确的步骤排序
- 步骤之间的依赖关系
- 每个阶段的验证
- 故障回滚路径

## 模式 2：多 MCP 协调

适用场景：工作流程跨多个服务时。

示例：从设计到开发的交接

```
Phase 1: Design Export (Figma MCP)
1. Export design assets from Figma
2. Generate design specifications
3. Create asset manifest

Phase 2: Asset Storage (Drive MCP)
1. Create project folder in Drive
2. Upload all assets
3. Generate shareable links

Phase 3: Task Creation (Linear MCP)
1. Create development tasks
2. Attach asset links to tasks
3. Assign to engineering team

Phase 4: Notification (Slack MCP)
1. Post handoff summary to #engineering
2. Include asset links and task references
```

关键技巧：

- 清晰的阶段划分
- 在多个 MCP 之间传递数据
- 进入下一阶段前进行验证
- 集中式错误处理

## 模式 3：迭代优化

适用场景：输出质量会随着迭代而提升。

示例：报告生成

```
Iterative Report Creation

Initial Draft
1. Fetch data via MCP
2. Generate first draft report
3. Save to temporary file

Quality Check
1. Run validation script: `scripts/check_report.py`
2. Identify issues:
 - Missing sections
 - Inconsistent formatting
 - Data validation errors

Refinement Loop
1. Address each identified issue
2. Regenerate affected sections
3. Re-validate
4. Repeat until quality threshold met

Finalization
1. Apply final formatting
2. Generate summary
3. Save final version
```

关键技巧：

- 明确的质量标准
- 迭代式优化
- 验证脚本
- 明确何时停止迭代

## 模式 4：上下文感知工具选择

适用场景，结果相同，但根据具体场景选择不同工具

示例，文件存储

```
Smart File Storage

Decision Tree
1. Check file type and size
2. Determine best storage location:
 - Large files (>10MB): Use cloud storage MCP
 - Collaborative docs: Use Notion/Docs MCP
 - Code files: Use GitHub MCP
 - Temporary files: Use local storage

Execute Storage
Based on decision:
- Call appropriate MCP tool
- Apply service-specific metadata
- Generate access link

Provide Context to User
Explain why that storage was chosen
```

关键技巧，

- 明确的决策标准
- 备用选项
- 选择的透明度

## 模式 5：领域特定智能

适用场景，你的技能能提供工具访问权限之外的专业知识。

示例，财务合规

```
Payment Processing with Compliance

Before Processing (Compliance Check)
1. Fetch transaction details via MCP
2. Apply compliance rules:
 - Check sanctions lists
 - Verify jurisdiction allowances
 - Assess risk level
3. Document compliance decision

Processing
IF compliance passed:
 - Call payment processing MCP tool
 - Apply appropriate fraud checks
 - Process transaction
ELSE:
 - Flag for review
 - Create compliance case

Audit Trail
- Log all compliance checks
- Record processing decisions
- Generate audit report
```

关键技巧，

- 将领域专业知识嵌入逻辑
- 行动前先合规
- 全面的文档追溯
- 清晰的治理



## 故障排除

### 技能无法上传

错误：“上传的文件夹中未找到 SKILL.md 文件”

原因，文件未完全命名为 SKILL.md

解决方案：

- 重命名为 SKILL.md（区分大小写）
- 验证方式：执行 `ls -la` 命令应能看到 SKILL.md 文件

错误：“无效的前置内容”

原因，YAML 格式问题

常见错误：

```
Wrong - missing delimiters
name: my-skill
description: Does things

Wrong - unclosed quotes
name: my-skill
description: "Does things

Correct

name: my-skill
description: Does things

```

错误：“无效的技能名称”

原因，名称包含空格或大写字母

```
Wrong
name: My Cool Skill

Correct
name: my-cool-skill
```

### 技能不会触发

症状：技能永远不会自动加载

修复：

修改您的描述字段。有关好 / 坏示例，请参阅描述字段。

快速清单：

它是否太通用？（“帮助项目”不起作用）

它是否包括用户实际会说的触发短语？

- 如果适用，它是否提到相关文件类型？

调试方式：

问克劳德：“你什么时候会使用【技能名称】技能？” 克劳德会引用描述。根据缺失的内容进行调整。

### 技能触发太频繁了

症状：不相关查询的技能加载

解决方案：

1. 添加负面触发词

```
description: Advanced data analysis for CSV files. Use for
statistical modeling, regression, clustering. Do NOT use for
simple data exploration (use data-viz skill instead).
```

## 2. 更具体一些

```
Too broad
description: Processes documents

More specific
description: Processes PDF legal documents for contract review
```

## 3. 明确范围

```
description: PayFlow payment processing for e-commerce. Use
specifically for online payment workflows, not for general
financial queries.
```

## MCP 连接问题

症状：技能加载成功但 MCP 调用失败

检查清单：

1. 验证 MCP 服务器是否已连接
  - Claude.ai: 设置 > 扩展 > [你的服务] 应显示“已连接”状态
2. 检查身份验证
  - API 密钥有效且未过期
  - 已授予适当的权限 / 作用域
  - 刷新 OAuth 令牌
3. 独立测试 MCP
  - 让 Claude 直接调用 MCP (不通过技能)
  - “使用 [服务] MCP 获取我的项目”
  - 若此操作失败，则问题出在 MCP 而非技能
4. 验证工具名称
  - 技能引用正确的 MCP 工具名称
  - 查看 MCP 服务文档
  - 工具名称区分大小写

## 指令未被遵循

症状：技能加载正常，但 Claude 不遵循指令

常见原因：

1. 指令过于冗长
  - 保持指令简洁
  - 使用项目符号和编号列表

将详细参考内容移至单独的文件中
2. 指令被隐藏
  - 将关键信息放在开头
  - 使用 ## 重要或 ## 关键标题
  - 必要时重复要点
3. 语言表达模糊

```
Bad
Make sure to validate things properly
```

```
Good
CRITICAL: Before calling create_project, verify:
- Project name is non-empty
- At least one team member assigned
- Start date is not in the past
```

高级技巧：对于关键验证，可考虑打包一个脚本，以编程方式执行检查，而非依赖语言指令。代码具有确定性，而语言解释则不具备这一特点。有关该模式的示例，可参考办公技能相关内容。

4. 模型“惰性”添加明确的鼓励语：

```
Performance Notes
- Take your time to do this thoroughly
- Quality is more important than speed
- Do not skip validation steps
```

注意：将此内容添加到用户提示中比在 SKILL.md 中更有效

## 大上下文问题

症状：技能运行缓慢或响应质量下降

原因：

- 技能内容过大
- 同时启用的技能过多
- 加载了所有内容而非逐步披露

解决方案：

1. 优化 SKILL.md 文件大小
  - 将详细文档移至 references/ 目录下
  - 链接至参考资料而非内联
  - 将 SKILL.md 控制在 5000 字以内
2. 减少启用的技能
  - 评估是否同时启用了超过 20 至 50 个技能
  - 建议选择性启用
    - 为相关功能考虑使用技能“组合包”

leeeeeee

第 6 章

# 资源与参考资料

# 资源与参考资料

如果你正在开发你的第一个技能，请先从《最佳实践指南》入手，然后根据  
需要参考 APT 文档。

## 官方文档

### Anthropic 资源：

- 最佳实践指南
- 技能文档
- APT 参考
- MCP 文档

### 博客文章：

- 智能体技能介绍
- 工程博文：为智能体赋能以适应现实世界
- 技能解析
- 如何为 Claude 创建技能
- 为 Claude Code 构建技能
- 借助技能优化前端设计

## 示例技能

### 公开技能仓库。

- GitHub: anthropics/skills
- 包含 Anthropic 开发的、你可自定义的技能

## 工具与实用程序

### 技能创建工具。

- 内置在 Claude ai 中，也可用于 Claude Code
- 可根据描述生成技能
- 进行评估并提供建议
- 用法：“帮我使用技能创建器构建一个技能”

### 验证。

- skill-creator 可以评估你的技能
- 询问：“查看这个技能并提出改进建议”

## 获取支持

### 技术问题。

- 通用问题：可前往 Claude 开发者 Discord 社区论坛

### 用于提交错误报告：

- GitHub 议题: anthropics/skills/issues
- 包含：技能名称、错误信息、重现步骤

# 参考 A：快速核对清单

使用此清单在上传前后验证你的技能。若想更快上手，可使用技能创建器技能生成初稿，然后对照此清单进行检查，确保没有遗漏任何内容。

## 开始前

确定 2-3 个具体的使用场景

- ☐ 已确定工具（内置工具或 MCP）
- ☐ 已阅读本指南和示例技能
- ☐ 规划文件夹结构

## 开发过程中

- ☐ 文件夹采用短横线命名法
- ☐ SKILL.md 文件存在（拼写完全一致）
- ☐ YAML 前置内容包含 `---` 分隔符
- ☐ 名称字段：使用短横线命名法，无空格，无大写字母
- ☐ 描述句含功能和适用场景
- ☐ 任何地方都不得出现 XML 标签（<>）
- ☐ 说明清晰且具有可操作性
- ☐ 命令格式外理 `_`
- ☐ 已提供示例
- ☐ 引用链接清晰 `_`

## 上传前

在明显任务上测试触发

测试对改写后的请求的触发情况

- ☐ 确认不会在不相关主题下触发

功能测试通过

- ☐ 工具集成正常（如适用）
- ☐ 压缩为 `.zip` 文件

## 上传后

- ☐ 在真实对话中测试

监控触发不足或触发过度的情况

- ☐ 收集用户反馈
- ☐ 优化描述和说明 `_`
- ☐ 在元数据中更新版本号 `_`

## 参考 B: YAML 前置内容

### 必填字段

```

name: skill-name-in-kebab-case
description: What it does and when to use it. Include specific
trigger phrases.

```

### 所有可选字段

```
name: skill-name
description: [required description]
license: MIT # Optional: License for open-source
allowed-tools: "Bash(python:*) Bash(npm:*) WebFetch" # Optional:
Restrict tool access
metadata: # Optional: Custom fields
 author: Company Name
 version: 1.0.0
 mcp-server: server-name
 category: productivity
 tags: [project-management, automation]
 documentation: https://example.com/docs
 support: support@example.com
```

### 安全说明

#### 允许:

- 任何标准 YAML 类型 (字符串、数字、布尔值、列表、对象)
- 空白元数据字段
- 长描述 (最多 1024 个字符)

#### 禁止:

- XML 尖括号 (< >) - 安全限制
- YAML 中的代码块行 (使用安全的 YAML 解析)
- 以 “claude” 或 “anthropic” 为前缀命名的技能 (保留)

# 参考 C：完整技能示例

用于展示本指南中各类模式的完整、可投入生产的技能示例：

- 文档技能 - PDF、DOCX、PPTX、XLSX 文档创建
- 示例技能 - 多种工作流模式
- 合作伙伴技能目录 - 查看来自 Asana、Atlassian、Canva、Figma、Sentry、Zanier 等各类合作伙伴的技能

这些代码仓库保持最新状态，并且包含了本文未涵盖的更多示例。你可以克隆它们，根据自身需求进行修改，并将其用作模板。





Claude

claude.ai